

Chapter 5 Network Layer: Control Plane

Introduction

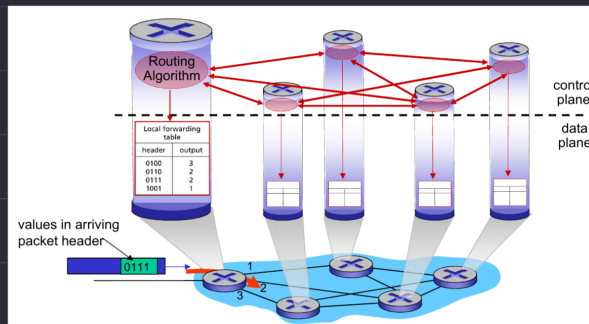
Network-layer functions

Data plane: forwarding

Control plane: routing

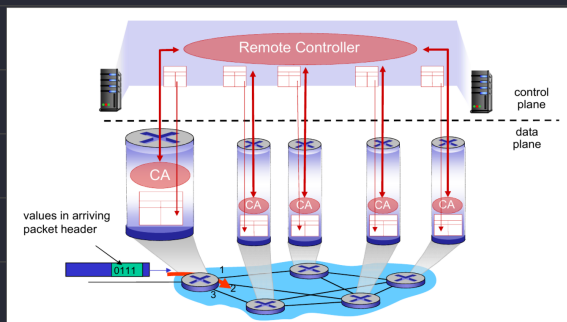
Two approaches to structuring network control plane

Per-Router Control: routing algorithm components in each and every router



Logically Centralized Control (software defined networking)

Remote controller computes, installs forwarding tables in router



Routing Protocols

Routing protocol goal: determine “good” paths

Path: sequence of routers packets traverse from source to destination

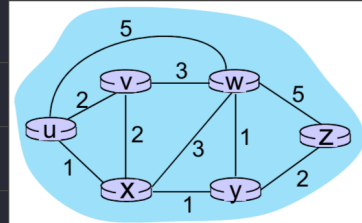
“Good”: least “cost”, “fastest”, “least congested”

Graph abstraction: link costs

graph: $G = (N, E)$

N : set of routers = $\{ u, v, w, x, y, z \}$

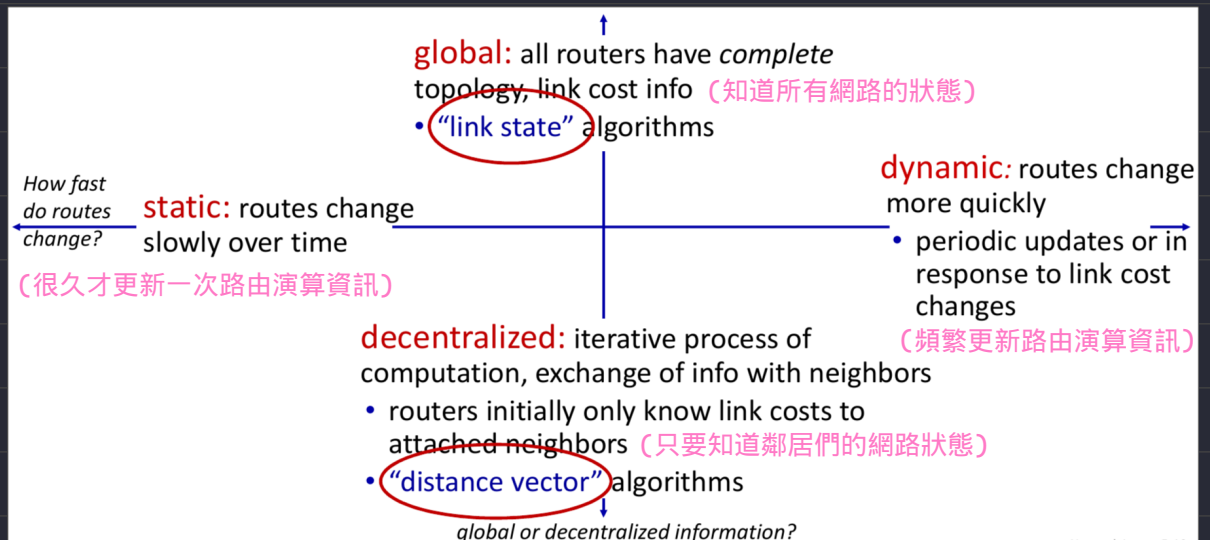
E : set of links = $\{ (u, v), (u, x), (v, x), (v, w), (x, w), (x, y), (w, y), (w, z), (y, z) \}$



$c_{w,z}$: cost (usually means distance) from w to $z = 5$

A pair of edge with no link between has infinity costs, e.g. $c_{u,z} = \infty$

Routing algorithm classification



Routing Protocols: Link State

Dijkstra's link-state routing algorithm

1. centralized: 通過 "link state broadcast" ^{flood} 讓所有 router 知道整個網路的線路狀態
2. 通過資訊更新 forwarding table
3. iterative: 每經過一個 router 就要算一次，網路上有 N 個節點就要迭代 N 次

Notation

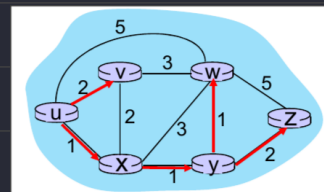
- D: 距離
1. $D(v)$: current estimate of cost least-cost-path from source to destination v
 2. $p(v)$: predecessor node along path from source to v (抵達v的前一個node的 path)
 3. N' : set of nodes whose least-cost-path definitively known

Algorithm

```
1 Initialization:
2  $N' = \{u\}$  /* compute least cost path from u to all other nodes */
3 for all nodes v
4   if v adjacent to u /* u initially knows direct-path-cost only to direct neighbors */
5     then  $D(v) = c_{u,v}$  /* but may not be minimum cost! */
6   else  $D(v) = \infty$ 
7
8 Loop
9   find w not in  $N'$  such that  $D(w)$  is a minimum
10  add w to  $N'$ 
11  update  $D(v)$  for all v adjacent to w and not in  $N'$ :
12     $D(v) = \min(D(v), D(w) + c_{w,v})$ 
13  /* new least-path-cost to v is either old least-cost-path to v or known
14  least-cost-path to w plus direct-cost from w to v */
15 until all nodes in  $N'$ 
```

Example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x	2, x	∞	∞
2	uxy	2, u	3, y	3, y	4, y	∞
3	uxyv	2, u	3, y	3, y	4, y	4, y
4	uxyvw	2, u	3, y	3, y	4, y	4, y
5	uxyvwz	2, u	3, y	3, y	4, y	4, y



1. 更新相鄰node的距離
2. 檢查如果經過另一個 node 會不會使距離縮短
3. 將最短的 node 加入 set

Usage

假如要到 w 就看 w column $\rightarrow (3, y) \rightarrow (2, x)$ y column $\rightarrow (1, u)$ x column $\rightarrow u, x, y$

震盪:

1. 如果所有路由器同時執行更新，發現有一側的路徑很短，全部湧入，導致該路線塞車
2. 原本路徑變得很空，所有路由器又同時發現改走，壅塞問題反覆平衡跳

Routing Protocols: Distance Vector

Based on Bellman-Ford (BF) equation

$$D_x(y) = \min_v \{c_{x,v} + D_v(y)\}$$

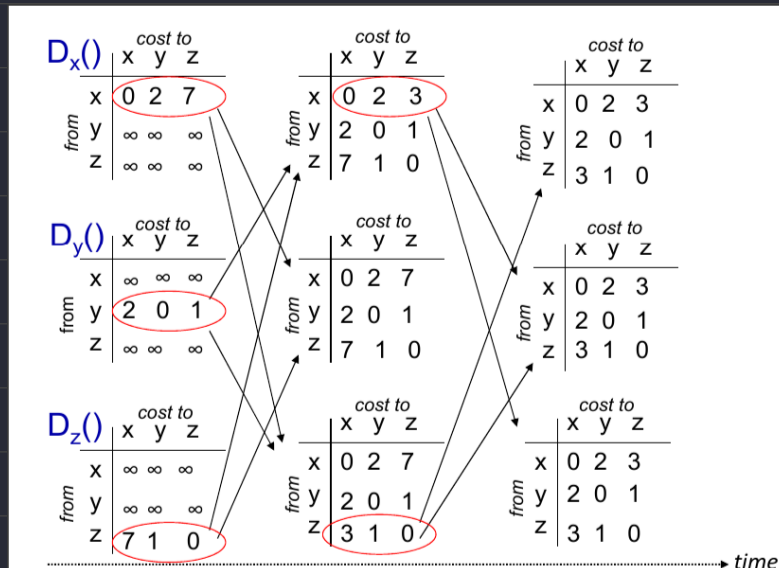
v 's estimated least-cost-path cost to y

direct cost of link from x to v

Steps (loop)

1. Wait for DV message from others or local cost changed
2. Recompute the DV value by the formula above
3. Notify neighbors if DV has changed

Examples:



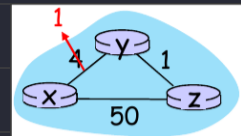
Link cost changes – oscillations (震盪)

Good news travels fast

t_0 : y detects link-cost change, updates its DV, informs its neighbors.

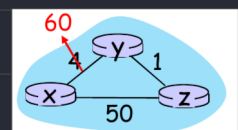
t_1 : z receives update from y, updates its table, computes new least cost to x, sends its neighbors its DV.

t_2 : y receives z's update, updates its distance table. y's least costs do *not* change, so y does *not* send a message to z.



Bad news travels slow:

- y sees direct link to x has new cost 60, but z has said it has a path at cost of 5. So y computes "my new cost to x will be 6, via z); notifies z of new cost of 6 to x.
- z learns that path to x via y has new cost 6, so z computes "my new cost to x will be 7 via y), notifies y of new cost of 7 to x.
- y learns that path to x via z has new cost 7, so y computes "my new cost to x will be 8 via y), notifies z of new cost of 8 to x.
- z learns that path to x via y has new cost 8, so z computes "my new cost to x will be 9 via y), notifies y of new cost of 9 to x.
- ...



Comparison of LS and DV algorithms

	Link State
Message complexity	$O(n^2)$ message sent
Speed of convergence	$O(n^2)$ algorithm, $O(n^2)$ messages, have oscillations (震盪)
Robustness	Router can advertise incorrect link cost

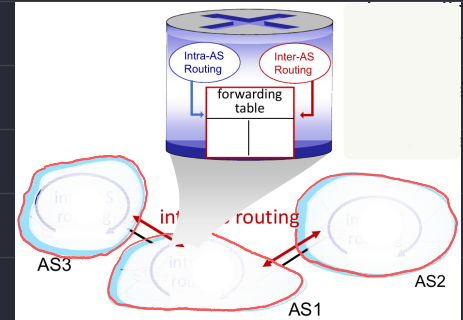
	Distance Vector
Message complexity	exchange between neighbors, convergence time varies
Speed of convergence	convergence time varies / routing loops (count-to-infinity)
Robustness	advertise incorrect path cost / error propagate thru network

Intra-ISP routing: OSPF

Interconnected ASes

範圍：Intra-AS

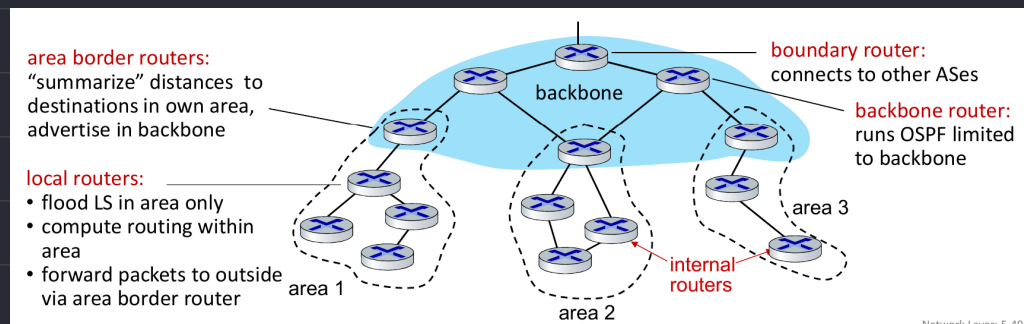
特性：都是自己人，追求極致效能



OSPF (Open Shortest Path First) routing

1. Advertise OSPF link-state to all router **directly over IP** (instead of TCP/UDP)
2. multiple link costs metrics(指標) possible: bandwidth, delay
3. each router has full topology, uses Dijkstra to compute forwarding table

Hierarchical OSPF



1. link-state advertisements flooded only in area, or backbone
2. each node has detailed area topology, only knows direction to reach other dest

Routing among ISPs: BGP

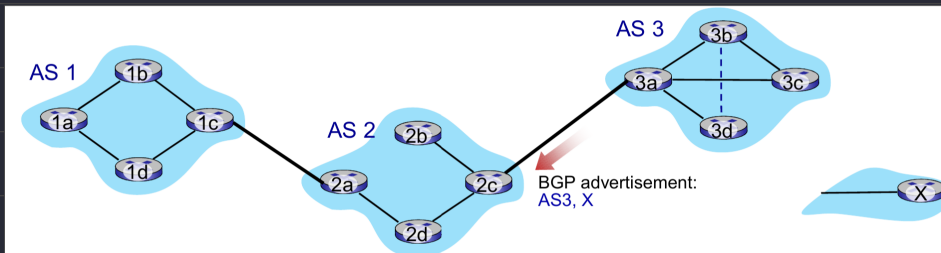
Internet inter-AS routing: BGP (Border Gateway Protocol)

eBGP (external): obtain subnet reachability information from neighboring ASes

iBGP (internal): propagate reachability information to all AS-internal routers.

BGP basics

BGP session: BGP routers ("peers") exchange BGP messages over semi-permanent TCP connection



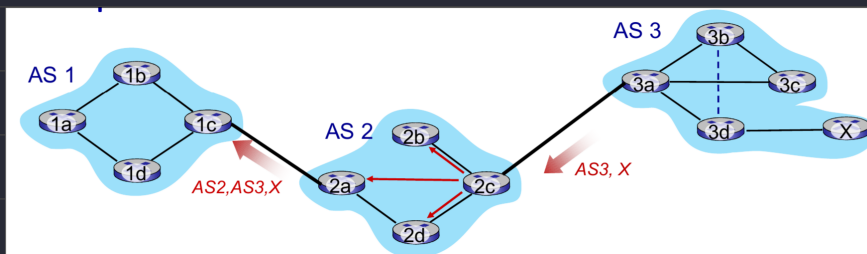
Policy-based routing (e.g. 打仗時兩國互不位彼此提供 routing 服務)

Receive advertisement according to the policy

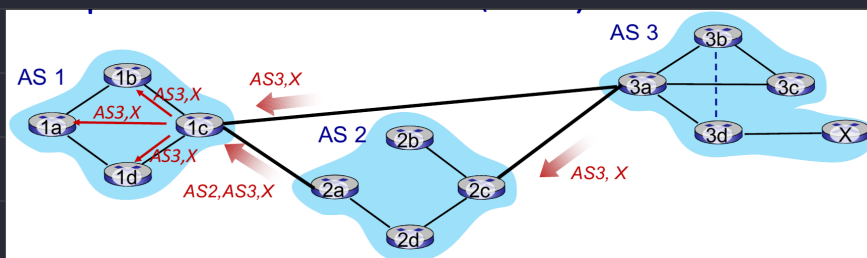
Decide which neighboring ASes to advertise BGP message according to policy

BGP path advertisement

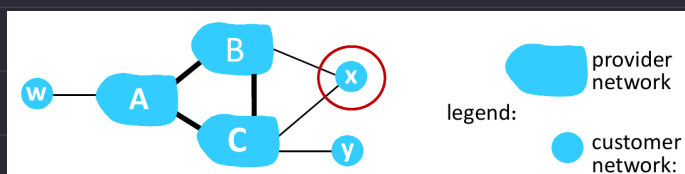
往上堆疊新的路徑訊息



收到多個可用的路徑



Achieving policy via advertisements



避免向 B 廣播 C 相關的路徑，反之亦然，即可避免自己成為 BC 之間的橋樑

Intra-AS vs. Inter-AS

Inter-AS	Intra-AS
Policy: No policy decisions (都是自己人)	管理誰可以經過這個網路與可以通向哪裡
Scale: Hierarchical routing (分層路由) saves table size, reduced update traffic	
Performance: Focus on performance	Policy dominates over performance

Hot potato routing

Only cares about intra-domain cost, not caring inter-domain.

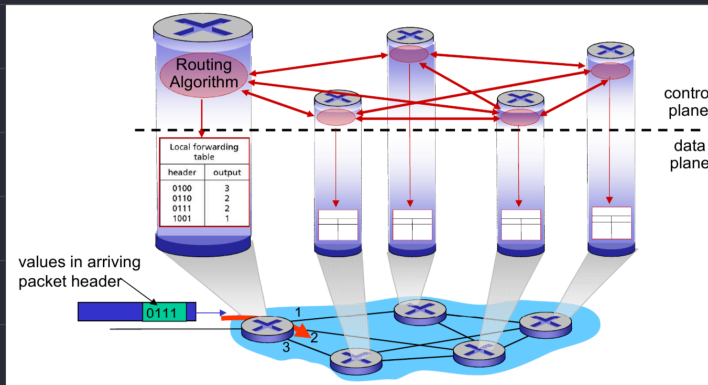
Choose the path has least intra costs, and don't care about inter costs

BGP vs. OSPF

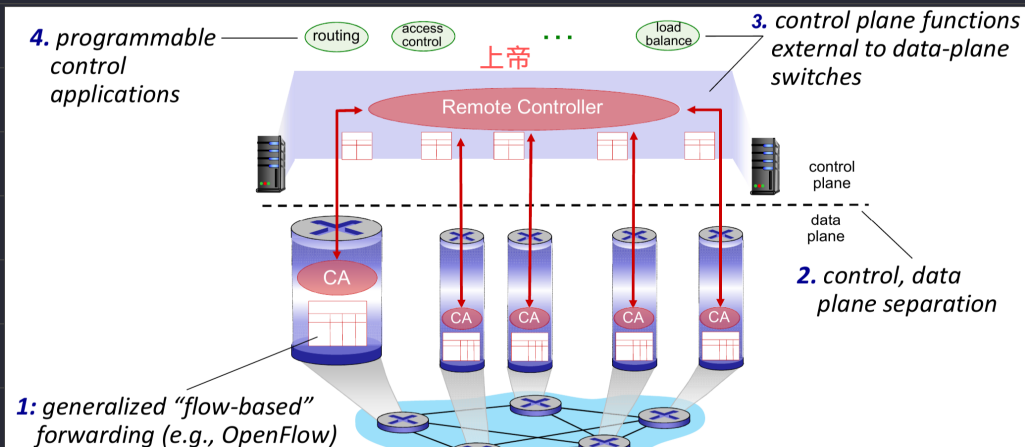
	BGP	OSPF
Protocol	TCP	IP
Focus	Policy	Performance
Circumstances	Internal / External	Internal

SDN control plane (Software defined networking)

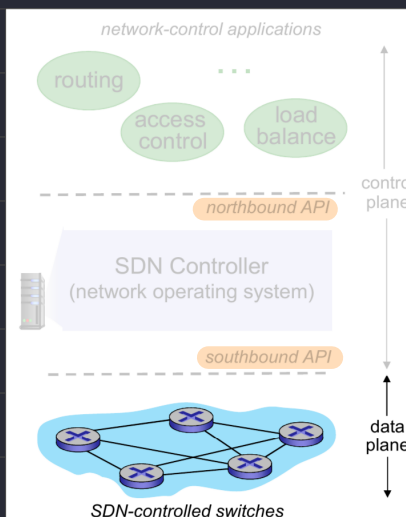
Per-router control plane 每個 Router 都有自己的演算法



SDN



Architecture



"brains" of control:

implement control functions

like network OS

interacts with

- control applications "above" via northbound API
- switches below via southbound API

API for table-based switch control

e.g. OpenFlow

Network Layer: 5-78 以後跳過

Internet Control Message Protocol (ICMP)

Get started

These type of message **directly** put in the IP datagram, not wrapped in TCP/UDP

ICMP message: type, code + first 8 bytes of IP datagram causing error

Traceroute and ICMP

Traceroute program uses ICMP to get the route path

1. Send packet to dest with TTL=1 → Get IP return from the first router
2. Send packet to dest with TTL=2 → Get IP return from the second router
- ...

Type	Code	description
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

Network management, configuration

Simple Network Management Protocol (SNMP)

SNMP is mostly used under many circumstances